Programming Project #1

CIS 2353 - Prof. John P. Baugh – Oakland Community College – OR

Objectives

- Design a custom class that models a real-world object (a game character loadout).
- Implement **equality** rules for objects (when two objects should be considered "the same").
- Implement **ordering/comparison** rules for objects (how objects should be ranked/sorted).
- Produce readable output for objects using a string representation method.
- Practice working with a **list/collection** of objects and basic testing in a driver program.

Scenario

In many games, a character's "loadout" includes a weapon, armor, and a set of modifiers (perks/enchantments). Your job is to model a loadout and support:

1. A readable string representation (for printing)
2. Equality comparison (are two loadouts equal?)
3. Ordering comparison (which loadout is stronger?)

Your approach will vary slightly depending on which programming language you choose (e.g., Java, Python, C++, C#).

Note that you must **only implement your project using a *single* programming language**. Multiple languages are allowed due to diverse student backgrounds and preferences.

Requirements

1. Create a Loadout class

Your Loadout class must contain the following fields:

A. Weapon Type

If your programming language supports enumerations (Java, C++, C#, Python), you **must** use them.

Only use strings if your chosen language does not reasonably support enums.

Valid values (weapon tier order matters later):

- Dagger
- Sword
- Axe
- Bow
- Staff

B. Armor Type

If your programming language supports enumerations (Java, C++, C#, Python), you **must** use them.

Valid values (armor tier order matters later):

- Cloth
- Leather
- Chain
- Plate

C. Modifiers (a list/collection of strings)

A collection holding **zero or more** modifier strings.

Valid modifiers (choose from this set only):

- CritChance
- AttackSpeed
- LifeSteal
- Poison
- Fire
- Ice
- Lightning
- DefenseBoost
- HealthBoost

Notes:

- The modifiers list may be empty.
- You should prevent duplicates.

2. Constructors/Initialization

Your class must support:

A. No-arg constructor / default initialization

Sets the following fields to the values:

- weapon = Dagger

- armor = Cloth
- modifiers = empty list

B. Full constructor / parameterized initialization
Allows the client to supply weapon, armor, and a list of modifiers.

3. Getters / Setters (or equivalent)
Provide accessors/mutators appropriate to your language.

4. addModifier(modifierName)
A method that adds a single modifier.

Rules:

- Must reject invalid modifier strings (not in the allowed list).

5. Required Behaviors

String Representation
The Loadout class must provide a string representation and be printable in a readable format.

Example style (you can vary the exact formatting):

```
Loadout:
  Weapon:  Sword
  Armor:   Leather
  Modifiers:
    CritChance
    Ice
    AttackSpeed
```

If no modifiers, it should print:

```
Modifiers: none
```

Equality Rule
Two loadouts are considered **equal** if ***all*** of the following are true:

- They have the **same weapon type**
- They have the **same armor type**
- They have the **same *number* of modifiers**

Important:

- You are NOT required to compare modifier names for equality, only the count.

Note:

- It is possible for two loadouts to be considered equal even if they do not have the same modifiers.
- This is intentional and mirrors real-world object comparison rules.

Ordering / Comparison Rule

You must define a consistent ordering so loadouts can be ranked or sorted.

Comparison rules (in this exact priority order):

1. **Compare by number of modifiers**
   - More modifiers = greater loadout
2. If modifier counts are equal, compare by **weapon tier**
   - Weapon strength order (lowest → highest):
     - Dagger < Sword < Axe < Bow < Staff
3. If weapon tier is equal, compare by **armor tier**
   - Armor strength order (lowest → highest):
     - Cloth < Leather < Chain < Plate
4. If all are equal, the loadouts are equal in ordering (return 0 / equivalent)

Driver / Demo Program

Create a small demo program (main/driver) that:

1. Creates at least **5** loadouts (mix default and custom)
2. Prints them
3. Tests **equality** on at least **3 pairs**
4. Tests **comparison** on at least **5 comparisons**
5. Places all loadouts in a list/array and **sorts** them, then prints the sorted result
   a. The sorting must rely on your Loadout comparison implementation (not manual comparisons written directly in the driver code).

new ArrayList<>(List.of("a","b"))

Collections.sort(loadouts);

Deliverables

- You will be submitting your assignment through **GitHub**
  - Ensure you have a GitHub account first (sign up if you don't – they're free)
  - You can use **GitHub for Desktop** (works on Windows and macOS!)
  - If you prefer, you can use Git Bash (command line) instead
- Instructions:
  - Click the provided submission link
  - Select the **Accept this assignment** button
    - This will generate an account-specific repo based on the provided template
  - Go into that repo by clicking the link
  - When you want to prepare the repo, go to **Code** button and click **Open with GitHub Desktop**

    - Make your commits and pushes to the main branch using GitHub Desktop (or command line, if you prefer)
- Your repo must include:
    - Source code and any relevant resource files
    - **At least 5 screenshots** of your program working
- When you are ready to be graded, submit the **link to your repository** through D2L

Rubric

Category	Criteria	Points
Class Design & Fields	`Loadout` class is correctly defined with weapon type, armor type, and modifiers collection. Required fields exist and are correctly typed.	15
	Weapon and armor types use enumerations when supported by the language.	5
Constructors & Initialization	Default (no-argument) constructor initializes weapon, armor, and modifiers exactly as specified.	5
	Parameterized constructor correctly initializes all fields from client input.	5
Modifiers Handling	`addModifier` correctly validates modifier names against the allowed list.	5
	Duplicate modifiers are properly prevented (ignored or rejected consistently).	5
String Representation	Loadout provides a readable string representation displaying weapon, armor, and modifiers.	10
	Output clearly handles the "no modifiers" case.	5
Equality Logic	Equality comparison correctly evaluates weapon type, armor type, and number of modifiers.	10
	Equality logic behaves consistently and correctly across tested pairs.	5
Ordering / Comparison Logic	Comparison logic follows the required priority order (modifiers → weapon tier → armor tier).	15
	Comparison results are consistent (reflexive, symmetric, transitive).	5
Sorting Requirement	Loadouts are placed into a collection and sorted using the implemented comparison logic (not manual driver code comparisons).	5
Driver / Demo Program	Demo program creates at least 5 loadouts (mix of default and custom).	5

	Program demonstrates equality tests, comparison tests, and sorted output clearly.	5
Code Quality & Submission	Code is readable, organized, and appropriately commented for the chosen language.	5
	Submission includes required screenshots and compiles/runs successfully.	5
Total		**100**

Language-Specific Hints

Depending on the programming language you have chosen, different hints may be useful.

Java Hints

- Implement Comparable<Loadout> and override:
  - `public int compareTo(Loadout other)`
- Override:
  - `public String toString()`
  - `public boolean equals(Object o)`
- Strongly recommended:
  - Also override `hashCode()` if you override `equals()` (especially if you ever put Loadouts into a `HashSet` / `HashMap`).
- Use `enums`:
  - `enum WeaponType { DAGGER, SWORD, AXE, BOW, STAFF }`
  - `enum ArmorType { CLOTH, LEATHER, CHAIN, PLATE }`
- Tip for tier ranking:
  - Use `enum` ordinal order (carefully), or create a method that returns an integer "tier value".

Python Hints

- Use `Enum` for weapon/armor (recommended):
  - `from enum import Enum`
- String representation:
  - implement `__str__` (and optionally `__repr__`)
- Equality:
  - implement `__eq__`
- Ordering:
  - implement `__lt__` (and optionally use `functools.total_ordering` to fill in others)
  - OR define a `sort_key()` method and sort using:
    - `sorted(loadouts, key=lambda x: x.sort_key())`
- Very clean approach:

    - define a tuple key like:
        - `(len(modifiers), weapon_tier, armor_tier)`
    - and compare tuples

C++ Hints

- Use `enum class` for weapon/armor (recommended).
- String representation:
    - create a `toString()` method
    - and/or overload `operator<<` for `std::ostream`
- Equality:
    - overload `operator==`
- Ordering:
    - implement `operator<` based on the comparison rules
    - Sorting: use `std::sort(vec.begin(), vec.end())` if `<` is defined
- Tip:
    - For tier ranking, write helper functions:
        - `int weaponTier(WeaponType w)` and `int armorTier(ArmorType a)`

C# Hints

- Use `enum` for weapon/armor.
- String representation:
    - `override public override string ToString()`
- Equality options:
    - `override Equals(object obj)` and `GetHashCode()`
    - OR implement `IEquatable<Loadout>`
- Ordering options:
    - implement `IComparable<Loadout>` with `CompareTo(Loadout other)`
    - or provide a custom `IComparer<Loadout>` for sorting
- Sorting:
    - `loadouts.Sort()` if `IComparable` is implemented
    - OR `loadouts.Sort(new LoadoutComparer())`